Programming Assignment 3: Risk & Q-Learning

Due: Friday 05/01/2026 @ 11:59pm EST

The purpose of programming assignments is to use the concepts that we learn in class to solve an actual real-world task. This time Collin has implemented the game engine (and did a far better job than I could). In this assignment you will be imlementing and training a Reinforcement Learning agent to play 2-6 player risk!

WARNING: There are **NO EXTENSIONS** for this assignment. This assignment has a **firm** due date of the last day of class!

Task 1. Reinforcement Learning via Sampling

Reinforcement Learning is a style of artificial intelligence where we aren't given input data and we aren't given ground truth. Instead, we are only given a world which our agent can interact with. The agent can observe the state of the world to whatever degree of observability is inherent to the world and interacts with the world by selecting actions. Upon selecting an action, the world will resolve that action internally and transition to a (potentially new) state which your agent can then observe.

This is problematic when we want our agent to **learn** how to select actions. Our only mechanisms for learning requires us to have some objective function to optimize, and that objective function needs some notion of ground truth (e.g. what is good gameplay). Since we don't have access to ground truth we have to **invent it** on the fly. This may sound strange but remember that if we were to simulate the game tree (for example MCTS) we could get a pretty decent notion of what is good and what is bad.

One important piece of information that we haven't had in the past is access to a **reward function**. You can think of a reward function as a way of steering the model using our human-intuition of what is good and what is bad. A reward function produces a value after an action is made. If this value is negative you can imagine shocking the model with some electricity (the magnitude of the value determines the severity of the shock). If the value is positive you can imaging giving the model a dopamine hit (again the magnitude controlling the severity). A value of zero is neutral. If we have a reward function we can use it in combination with any performance values we get by expanding the game tree to produce ground truth with which we can train our model. The reward function is sometimes provided as part of the world, othertimes whoever designs the agent is responsible for engineering it. We will choose the latter option for this assignment (e.g. you have to engineer it).

In this assignment we are going to play a bunch of games and record what the state of the game looked like, the actions chosen by the agent, and the reward values we received along the way. Formally a triple of (s, a, s') is called a **transition** (also called a **full**-transition) which we distinguish from a **half**-transition (s, a). Recording the states of the game and the actions selected by the agent produces **many** full and **many** half transitions with which we are going to use to generate our ground truth and subsequently train our model with. This requires some engineering decisions:

1. What senses should the agent have? The agent must perceive the state of the world in order to select an action.
2. How should the agent choose actions?
3. What should the reward function look like? There are three flavors of reward functions, and you get to choose in this assignment:
   - $R(s)$. Pick this option if you personally like reasoning about states in isolation. Things like "the agent is doing X well in this state so it should get a reward of Y."

- $R(s, a)$. Pick this option if you personally like reasoning about the choice of action in specific states. Things like "the agent chose this action in this state and that was a good/terrible decision so pleasure/punish the agent Y amount."
- $R(s, a, s')$. Pick this option if you personally like reasoning about how good specific actions in specific states are knowing how that action resolved. Things like "the agent chose this action in this state and that action resolved this way. Since it resolved in a good/bad way we should pleasure/punish the model in a corresponding way."

With these engineering decisions made, we are going to employ supervised learning techniques to learn what actions to make in what states. The goal here is to learn a **policy**: a function that maps states to the (optimal) action that should be taken. We will do this with the following paradigm:

1. Play a bunch of training games. The point of these games are to generate training data in the form of (state, reward, next_state) transitions. Normally we would wait for these training games to end before we convert these transition into a supervised learning dataset and update our model but this strategy does not work in risk. The reason why is that it is hard to end a game in risk: you have to play well in order to eliminate the other players (or wait an extremely long time via random action selection). In other games like tetris or pokemon, the game naturally wants to end: if you play poorly the game will end sooner rather than later. In risk the game could go on forever (or at least extremely long in practice) unless you know what you're doing. So if we use the normal scheme to wait until the training games complete before updating our model we may end up waiting an extremely long time. Instead we are going to update our model during the middle of training games (configured via a hyperparameter). This makes the process more noisy but in practice will lead to your agent learning faster (hopefully).

   An important detail when playing training games is to sometimes ignore the policy. This is obviously a bad idea if we happen to know that our policy is optimal. This is less of a bad idea if we happen to know that our policy sucks. The tricky part is that we don't know whether our policy is optimal or not. We can at least make an educated guess: when our agent is brand new (e.g. just "born"), the policy is randomly initialized, so it probably sucks. The older the agent gets, the more experience it has been trained on, so the chances the policy is good improves (terms and conditions may apply like overfitting, etc). So, we want to engineer some randomness into whether we listen to the policy when choosing an action. Many people engineer this randomness to be a function of time and make the probability we ignore the policy decrease as a function of age (just like simulated annealing).

2. After the training games complete and our model is updated, we want to see how well it performs. So we play a bunch of evaluation games where the model is frozen and any random exploration is disabled. The purpose of these games is not to record transitions and update the model but instead to evaluate how good the current policy performs.

Solving a Reinforcement learning problem like this is nice because it is guaranteed to learn the optimal model in the limit of infinite data. The downside to solving the problem like this is because of the practicality of "learning in the limit of infinite data." Sure, the model is guaranteed to get better eventually, the question is practically how long does it take? Training a model like this can take **ages**, its up to the quality of our engineering decisions and luck!

Task 2. Reinforcement Learning with Q-learning

Q-learning, as we will see in lecture, is a specific flavor of Reinforcement learning. In Q-learning, the model learns whats called a *Q-function*, which guesses how good an action is in a state $Q(s, a)$. If our model was optimal, then whenever our agent is in a state, we could calculate the best action to do via

$$a^* = \underset{a \in Actions(s)}{\arg\max} \; Q(s, a)$$

Q-learning tells us how to convert the training data generated from samples of transitions into a supervised learning dataset. Most of the work occurs in generating the ground truth. If we have a bunch of transitions (s, a, s') from playing the game (s was a state, a was the action taken in state s, and s' was the resulting state of applying a in s), then we generate ground truth using whats called the *Bellman* equation (for Q-functions):

$$Q(s, a) = R(s, a) + \gamma \max_{a' \in Actions(s')} Q(s', a')$$

Reminder that there are multiple flavors of rewards: you don't have to choose $R(s, a)$ like we've done here. The Q-Bellman equation just tells us what Q-value our model *should have* predicted, so we can use it as ground truth to train our model against. This is technically called **temporal difference learning** and we'll talk about it a bunch in class. The tl;dr is that we can apply this equation to generate our ground truth and then we learn a model to minimize the following loss function:

$$\mathcal{L}(\hat{\vec{q}}, \vec{q}_{gt}) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{q}^{(i)} - q_{gt}^{(i)} \right)^2$$

One note here is that our ground truth is going to be calculated slightly differently as an edge case of multiple players. In risk you get to make multiple actions per turn, so the transition of the **last** move of your turn will have half of the transition in your turn and the next state s' in **another player**'s turn. This is problematic when using the Bellman equation above which assumes **you** have agency in state s'. When we have transitions like this, we don't use the Bellman equation to generate the ground truth. Instead we use whats called the State Action Reward State Action (SARSA) ground truth which does not assume you have agency in s': it instead just repeats the chosen action a in s':

$$Q(s, a) = R(s, a) + \gamma Q(s', a)$$

This may feel weird, but the intention is to **prevent** your model from accidentally learning that it gets to make another move after your turn ends (which it is not allowed to do). This can cause your model to learn weird things if it incorrectly believes that after emitting a turn-ending move it gets to go again! So to be clear we use the following piecewise function to calculate ground truth:

$$Q(s, a) = \begin{cases} R(s, a) + \gamma \max_{a' \in Actions(s')} Q(s', a') & \text{you have agency in } s' \\ R(s, a) + \gamma Q(s', a) & \text{otherwise} \end{cases}$$

Task 3. Reinforcement Learning with Neural Networks

Reinforcement learning requires us to learn, in the case of q-learning, a single Q-value per (state, action) pair. This mapping is not feasible to store as a table for all but the simplest toy problems. For most problems (Pokemon included), there are simply too many states for us to allocate a single float per entry. When we encounter a scenario like this, we often use a neural network in place of the tabular function. Neural networks learn functions, so we can use them in this setting to learn the function while skipping the memory requirements for an exhaustive table.

Like all ML models, neural networks learn a kernel function $K(\vec{x}, \vec{y})$: a similarity function. This kernel is both a blessing and a curse. A kernel is a blessing because, if you see an example (that you've never seen before), you can leverage the knowledge that you've learned from similar examples (the kernel tells you how similar things are). A Kernel is a curse because of how it is constructed. A Neural Network learns a kernel through its parameters. If we change a single parameter (i.e. tweak one of the

many parameters inside the network), we change the **entire** kernel and therefore we change how the network views **every** example it will ever see. This means that if we try to improve performance on a single example, we can potentially destroy the knowledge that the network currently has contained within it. It is for this reason that when we train neural networks, we don't optimize for a single example: we take the average gradient across a batch of examples.

While we technically should update our network as soon as a transition occurs, we will instead aggregate them in a buffer called a *replay buffer*. The purpose of the replay buffer is to account for the curse of the kernel: if we updated our network with a transition the second we observe it, we would be optimizing for a single example and run the risk of our network forgetting all the knowledge it has already learned. So, instead of updating the network as soon as transitions occur, we will collect them into the replay buffer, and use this collection of transitions to update the model. To be clear, a replay buffer is a hack: this is a bandaid solution that does not solve the underlying problem. A more meaningful approach would be to address neural networks treating each example as i.i.d, but thats a topic for another time and is an active area of research.

Therefore, to be more accurate, we should update the training paradigm from the earlier sections. Training is organized into cycles. Each cycle has the following sections, which are executed in order:

1. Play a bunch of training games that populate the replay buffer. Normally we would keep our model frozen during this phase but for risk we will sometimes convert a chunk of the replay buffer into training data and update our model.

2. Normally we would then convert the replay buffer into a supervised learning dataset (using our ground truth equation equation from earlier). Train the neural network on this dataset. We are skipping this phase in risk and instead are combining this phase with the previous one.

3. Play a bunch of evaluation games with the updated (and now frozen) neural network, the sole purpose of which is to evaluate the performance of the network.

This cycle is repeated an enormous amount of times. Eventually, the network will get really good, and therefore our agent will get really good.

Task 4. Copy Files

Please copy the files from the downloaded lab directory to your cs440 directory. You can just drag and drop them in your file explorer.

- Copy `Downloads/risk/lib/*.jar` to `cs440/lib/*.jar`.
  There are a few `.jar` files in here and you will need all of them.
- Copy `Downloads/risk/src/pas` to `cs440/src/pas`.
  This directory contains our source code `.java` files.
- Copy `Downloads/risk/risk.srcs` to `cs440/risk.srcs`.
  This file contains the paths to the `.java` files we are working with in this lab. Just like last lab, files like these are used to speed up the compilation process by preventing you from listing all source files you want to compile manually.
- Copy `Downloads/risk/doc/pas` to `cs440/doc/pas/risk`. This is the documentation generated from the custom jarfiles we made and will be useful in this assignment. After copying, if you double click on `cs440/doc/pas/risk/index.html`, the documentation should open in your browser.

Task 5. Test run

If your setup is correct, you should be able to compile and execute the given template code. You should see a window appear showing a three player game of risk.

```
# Mac, Linux. Run from the cs440 directory.
javac -cp "./lib/*:." @risk.srcs
java -cp "./lib/*:." edu.bu.pas.risk.SingleGameEval

# Windows. Run from the cs440 directory.
javac -cp ./lib/*;. @risk.srcs
java -cp ./lib/*;. edu.bu.pas.risk.SingleGameEval
```

By default, three agents will be created, two `StaticAgent`s (which do nothing) and one `RandomAgent` (which chooses uniformly-random actions).

Task 6. Command Line Arguments

You are free to change the agents controlling each player as well as the number of players in the game. We support 2-6 players in risk. If you wish to do this be sure to look at the command line arguments available which you can do with (here is the Mac/Linux version):
`java -cp ./lib/*;.  edu.bu.pas.risk.SingleGameEval`

There are multiple executables within risk all within the package `edu.bu.pas.risk`. The executable mentioned so far `SingleGameEval` does exactly that, it plays a single game of risk where each model is frozen. This executable can also render the game for you. There is a `HumanAgent` so you can play against agents yourself!

The main executable you will use the most is `SequentialTrain`. This executable is how you will train your agent. While the other executables render by default (the rendering can be turned off), there is no rendering available for `SequentialTrain`. As discussed in lecture and detailed in the earlier sections, the purpose of `SequentialTrain` is to launch a massive "training run" where you iteratively

fit a neural network by collecting lots of experience playing the game, coverting that experience into a supervised learning dataset, and using that dataset to fit the model. This executable has quite a few arguments: some have to do with training hyperparameters, some let you choose how you train the network, some manage files, etc. Trigger the help string to see what is available!

Task 7. Decisions in Risk

One of the interesting things about risk (at least in our implementation of risk) is that there are two decision points: placing armies in territories and making actions. In order to capture these two decision points your agent is going to have **two** q-functions inside of it not just one. However, the two decision points require analyzing similar information: for intance they both depend on the state of the game in order to make effective choices. We have chosen to model this with two neural networks where some layers are shared across both models. As we've discussed in lecture (or will discuss), neural networks are broken up into two logical chunks: an **encoder** and a **decoder**. The encoder's job is to learn how to convert the input sample into a meaningful vector (called an **embedding**) while the decoder's job is to learn how to conver that embedding into meaningful decisions. Our neural network has a shared encoder which embeds the state of the game and then two separate decoders: one decoder for deciding what territories to place armies at (called the **placement decoder**) and another for deciding what actions to take (called the **action decoder**). I highly suggest you take a look at the `edu.bu.pas.risk.agent.IAgent` interface which explains how turns work.

Task 8. `My*SensorArray` classes

There are three files in directory `src/pas/risk/senses/`. The purpose of each class is to turn the corresponding data type into a feature vector which we can pass to the neural network. You will need to decide what sensors to equip your sensor array with and use the `getSensorValues` method to actually convert into a row vector. You will need to ask yourself the following questions:

1. What descriptive measurements should my model perceive about this datatype? Not all measurements are equally good: some measurements are more effective "summarizations" (e.g. draw better equivalance class boundaries) than others.

2. How can I calculate these measurements in a meaningful way? You want to avoid confusing the model: just like you can overstimulate a person you can also overstimulate the model.

Whatever sensing choices you make, you will need to implement that functionality in these files.

Task 9. `My*RewardFunction` classes

There are two files in directory `src/pas/risk/rewards/`. The purpose of each class is to provide a meaningful punishment/pleasure signal which we will incorporate into the ground truth. Since the model has two decoders (also called decoder **heads**) and can predict two q-values we need two reward functions. The purpose of a single reward function is to provide human-crafted feedback about how well the model is making the corresponding decision. As mentioned in an earlier section, you have a choice of which flavor of reward function you want to implement:

- $R(s)$. Pick this option if you personally like reasoning about states in isolation.

- $R(s, a)$. Pick this option if you personally like reasoning about the choice of action in specific states.

- $R(s, a, s')$. Pick this option if you personally like reasoning about how good specific actions in specific states are knowing how that action resolved.

Each of these flavors is implemented with a different method inside your reward function class. You are welcome to design multiple kinds of rewards and then combine them together: just like heuristics there are typically more than one way of measuring the "quality" of decisions/states. If you do this, the vocab term for this is called reward *shaping*.

Task 10. `class RiskQAgent`

Please take a look at the file `src/pas/risk/agent/RiskQAgent.java`. The purpose of this class is to wrap around your neural network instance. You will need to finish the following methods:

1. `initModel()`. This method instantiates your dual decoder neural network. See the starter code for an example. We are only using feed-forward neural networks in this class. Remember that decoders produce a single output each (e.g. a q-value) and the dimensionality of the input vectors is controlled by your sensor arrays.

2. `createStateSensors`/`createActionSensors`/`createPlacementSensors`. These methods are for instantiating your sensor arrays.

3. `createActionReward`/`createPlacementReward`. These methods are for instantiating your reward functions.

4. `getExplorationPlacement`. This method is responsible for picking a territory to place an army at when we choose to ignore the model.

5. `getExploration*Action*`. Theses methods are responsible for picking an action when we choose to ignore the model. There is one method per phase of a turn in risk. See the documentation for `IAgent` and `FSMAgent` for more details.

6. `shouldExplorePlacementPhase`. This method is responsible for choosing whether or not to listen to the model or to get a placement action via calling `getExplorationReplacement`.

7. `shouldExplore*MovePhase`. Theses methods are responsible for choosing whether or not to listen to the model or to get an action via calling the corresponding `getExploration*Action*` method.

Note that the implementation of risk has a listener system, so you are free to read the `IAgent` and `RiskListener` documentation to see what methods you can override to hook into this system. Note that some of ancestor classes of your `RiskQAgent` have implementations of these so if you choose to provide your own listeners don't forget to call the `super` version inside your version!

Task 11. Training

To train your agent, you should develop your code locally and make sure that it doesn't error, and that it measures quantities the way you intend them to be measured. Once you have that working, you will want to train your agent on tens of thousands, if not hundreds of thousands, if not millions of games. To do this, we are giving you access to the SCC. We will provide you a tutorial of how to use the SCC in lecture.

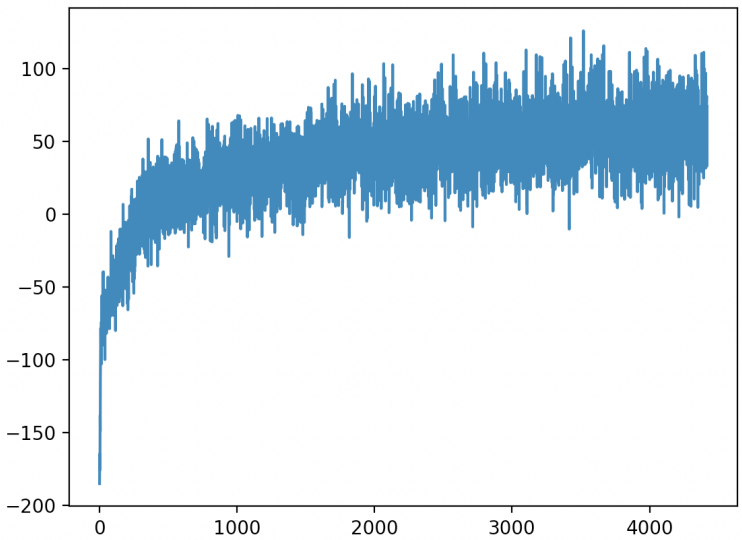
Task 12. Checking on Performance

When training your agent (which will take days), you will want to check in on it periodically. My code will, after every cycle, save your neural network to disk in a directory called `params`. You are free to change the destination of the saved models, and on the SCC I encourage you to save your models to the `projectnb` directory. When should you stop training, and which version of your network is the

best? These are questions answered by the evaluation section of every cycle. The average trajectory utility is printed out after every cycle: this is information you will want to keep and to plot. To do so, when it is time for you to launch your agent's massive training run, you should run it like so (only shown for linux since I am assuming you'll be running this on a linux machine):

```
java -cp "./lib/*:." edu.bu.pas.risk.SequentialTrain <OTHER_ARGS> | tee my_logfile.log
```

The `| tee my_logfile.log` part of this command will pipe stdout to a file called `my_logfile.log` which will exist in the same directory that the shell is in when the command was run (again I would recommend specifying a path to the `projectnb` directory). Periodically, you can download your `my_logfile.log` to your laptop, parse the file and plot the average trajectory utility as a function of time. The way you will know your model is learning is if you get something that looks like this:



The x-axis is the number of cycles that the model has run for (this curve shows 4000 cycles), and the y-axis is the average trajectory utility. Note that your curve may look very different than this and thats ok. What you want to see is that your model is improving over time. This curve is called a **learning curve**.

Task 12. Performance (100 points)

Risk supports 2-6 players, and the autograder will be checking that you can play good risk against various number of players. You will be turning in your code and a trained model. Inference is cheap so the autograder will be playing many games against a plethora of opponents designed by us. Some of these opponents we will produce as time goes on, some we will keep hidden from you. Your goal is to win as many games as you can.

Task 13. Extra Credit (50 points)

We are in the process of implementing some opponents that play really good risk. We are working on designing agents that collaborate together and will gang up on you. If you can defeat our collaboration agenst at all we will award you full extra credit.

Task 14. What to Submit

Please submit all of the provided `.java` files. If you make more `.java` files I cannot guarantee that I will place them in the correct directories so I will not use them at all. As your model trains, my

code will write the parameters of the model to files after every cycle. You need to pick one of these files (ideally the file that corresponds to the model that has the best performance), rename it to `params.model`, and submit it along with your code on gradescope. Reminder that this filename is case sensitive and must be **exactly** `params.model` for the autograder to recognize it.

Task 15. Tournament Eligibility

In order for your submission to be eligible in the tournament, your submission must satisfy all of the following requirements:

- Your submission must be on time.
- You do not get an extension for this assignment.
- Your agent compiles on the autograder.
- Your agent can win more than 50% against some of our opponent agenst (not the collaboration agents) that we will specify later.